

RepoLens AI

AI Codebase Archaeologist

Hackathon Project Documentation

Technology: React + FastAPI + Python AST + AI

Deployment approach: Local / LAN demo only (AWS excluded to avoid cost)

Prepared for a 4-member team | 8–12 hour hackathon MVP

1. Project Overview

RepoLens AI is an AI-powered codebase exploration and onboarding assistant. It accepts a GitHub repository URL or a local repository, analyzes supported source files, extracts module and dependency relationships, creates an architecture map, identifies important modules and possible entry points, and answers developer questions using repository-grounded context.

The project is based on the hackathon problem statement “The AI Codebase Archaeologist,” which asks teams to help developers understand unfamiliar repositories and identify where changes should be made.

2. Problem Statement

Developers joining an existing or legacy project can spend considerable time understanding the codebase. Files may be scattered across folders, dependencies may be difficult to trace, and important entry points may not be obvious. RepoLens AI reduces the initial exploration effort by providing a visual and conversational view of the repository.

3. Proposed Solution

- Accept a public GitHub repository URL or a prepared local repository.
- Analyze supported source files using static code analysis.
- Extract imports, functions, classes, modules, and relationships.
- Generate an interactive architecture/dependency graph.
- Identify important modules and likely entry points.
- Answer questions such as: “Where is authentication implemented?”
- Return relevant file paths, symbols, and limitations with AI answers.

4. Project Objectives

- Reduce the time required to understand an unfamiliar codebase.
- Provide a visual representation of module and import relationships.
- Help developers locate files and functions relevant to a change.
- Combine deterministic static analysis with AI-based explanations.
- Provide transparent answers grounded in repository context.
- Deliver a working, focused MVP within 8–12 hours.

5. MVP Scope

The team should focus on the following four core features:

Feature	Description	Priority
Repository Analyzer	Reads a repository, scans supported files, and extracts code metadata.	Must have
Architecture Map	Displays files/modules and import relationships as a graph.	Must have
AI Codebase Q&A	Answers repository questions using selected code context.	Must have
Modules & Entry Points	Shows important files, functions,	Must have

	classes, and possible entry points.	
--	-------------------------------------	--

Recommended scope: support Python repositories first. Add JavaScript/TypeScript only if the main MVP is stable.

6. Functional Features

Repository Input: Enter a GitHub URL or use a prepared local repository. Validate the input before analysis.

Source File Discovery: Traverse folders while excluding .git, virtual environments, build folders, and irrelevant files.

Static Code Analysis: Use Python AST to extract imports, functions, classes, and line-level symbols.

Dependency Extraction: Identify relationships between local project modules using import statements.

Architecture Visualization: Show nodes for files/modules and edges for import relationships.

Module Explorer: Click a node to view imports, functions, classes, and a short summary.

Entry Point Detection: Identify likely entry files such as main.py, app.py, or files containing execution guards.

AI Q&A: Answer questions using repository context, with relevant file references.

Error Handling: Display parsing errors, unsupported files, invalid repositories, and unavailable AI responses.

7. Non-Functional Requirements

- The application should be easy to demonstrate from one laptop.
- The frontend should provide clear loading, success, empty, and error states.
- The backend should not crash when one file contains syntax errors.
- API keys must never be committed to GitHub or exposed in React code.
- AI responses should clearly state when repository context is insufficient.
- The demo should work locally even if cloud deployment is not available.
- The system should place limits on repository size, file size, and AI context size.

8. System Architecture

The application contains three primary layers:

Layer	Technology	Responsibility
Frontend	React + Vite + React Flow	Repository input, dashboard, graph, module details, and chat UI.
Backend	FastAPI + Python	Repository ingestion, AST analysis, graph creation, and API endpoints.
AI/Data	LLM API + extracted repository context	Summaries, repository-grounded Q&A, and file references.

9. Data Flow

1. User enters a GitHub repository URL or selects a prepared local repository.
2. React sends the request to the FastAPI backend.
3. The backend downloads or reads the repository.
4. The analyzer discovers supported files and parses Python source code.
5. The system extracts modules, imports, functions, classes, and possible entry points.
6. The graph builder creates nodes and edges.

- 7. The Q&A service retrieves relevant repository context.
- 8. The AI generates an answer with file references and limitations.
- 9. React displays the graph, analysis results, and answer.

10. Technology Stack

Layer	Technology	Purpose
Frontend	React + Vite	Build the web interface.
UI / Graph	React Flow + CSS/Tailwind	Render architecture and dependency relationships.
Backend	FastAPI	Expose REST APIs and coordinate services.
Static Analysis	Python ast	Extract imports, classes, functions, and syntax metadata.
Repository Access	Git / GitHub API	Read public repositories or prepared local data.
AI	LLM API	Generate explanations and repository-grounded answers.
Storage	In-memory objects or JSON	Store MVP analysis results during a demo.
Version Control	Git + GitHub	Team collaboration and source control.
Deployment	Localhost / LAN	Avoid AWS cost and keep a reliable demo environment.

11. Team Roles and Responsibilities

Member	Role	Responsibilities	Deliverable
Member 1	Frontend Developer	Dashboard, repository input, graph UI, module panel, chat interface.	Working React dashboard
Member 2	Backend Developer	FastAPI routes, repository ingestion, file traversal, API response models.	Working analysis API
Member 3	AI & Code Analysis	Python AST analyzer, dependency extraction, context retrieval, LLM prompt and Q&A.	Working analysis and AI pipeline
Member 4	Integration & Presentation	Local setup, integration testing, environment variables, demo script, presentation.	Stable integrated demo and presentation

All members should integrate early. Do not wait until the final hour to combine frontend, backend, and AI work.

12. Project Folder Structure

```
repolens-ai/  
├── frontend/  
│   └── src/
```

```

├── components/
│   ├── RepoInput.jsx
│   ├── ArchitectureGraph.jsx
│   ├── ModuleList.jsx
│   ├── EntryPoints.jsx
│   └── ChatPanel.jsx
├── pages/
│   └── Dashboard.jsx
├── services/
│   └── api.js
├── App.jsx
├── main.jsx
├── package.json
├── .env.example
├── backend/
│   ├── app/
│   │   ├── main.py
│   │   └── api/
│   │       └── routes.py
│   ├── services/
│   │   ├── repository_service.py
│   │   ├── analyzer.py
│   │   ├── graph_builder.py
│   │   └── ai_service.py
│   ├── schemas/
│   │   └── models.py
│   └── utils/
│       └── security.py
├── requirements.txt
├── .env.example
├── sample-repo/
├── docs/
│   ├── architecture.md
│   └── demo-script.md
├── README.md
└── .gitignore

```

13. API Design

Method	Endpoint	Purpose	Input
GET	/health	Check backend status	None
POST	/api/analyze	Analyze a repository	repo_url or local repository reference
GET	/api/repository/{id}	Retrieve analysis results	Repository ID
GET	/api/repository/{id}/graph	Retrieve graph data	Repository ID
POST	/api/chat	Ask a question about the repository	repository_id + question

14. Example API Requests

Analyze repository request:

```
POST /api/analyze
```

```
{
  "repo_url": "https://github.com/example/sample-python-project"
}
```

Example analysis response:

```
{
  "repository_id": "repo_001",
  "language": "python",
  "total_files": 15,
  "modules": [],
  "dependencies": [],
  "entry_points": [],
  "graph": {
    "nodes": [],
    "edges": []
  }
}
```

Ask a question:

```
POST /api/chat
```

```
{
  "repository_id": "repo_001",
  "question": "Where is user authentication implemented?"
}
```

The response should include an answer, relevant files, supporting symbols, and a confidence or limitation note.

15. Python AST Analyzer

The Python `ast` module provides a practical way to inspect Python source code without relying entirely on AI. The MVP analyzer should extract imports, functions, and classes and return structured JSON.

```
import ast
from pathlib import Path

def analyze_python_file(file_path: str):
    path = Path(file_path)

    source = path.read_text(
        encoding="utf-8",
        errors="ignore"
    )

    tree = ast.parse(source)

    imports = []
    functions = []
    classes = []

    for node in ast.walk(tree):
        if isinstance(node, ast.Import):
            for alias in node.names:
                imports.append(alias.name)

        elif isinstance(node, ast.ImportFrom):
            if node.module:
```

```

        imports.append(node.module)

    elif isinstance(node, (ast.FunctionDef, ast.AsyncFunctionDef)):
        functions.append(node.name)

    elif isinstance(node, ast.ClassDef):
        classes.append(node.name)

    return {
        "file": str(path),
        "imports": sorted(set(imports)),
        "functions": sorted(set(functions)),
        "classes": sorted(set(classes))
    }

```

Recommended improvements:

- Add line numbers for functions and classes.
- Identify imports between local project modules.
- Skip .git, virtual environments, build folders, and large irrelevant files.
- Catch syntax errors and continue processing other files.
- Add limits for file size and total repository size.

16. Architecture Graph

The graph uses nodes and edges:

- Nodes represent files or modules.
- Edges represent import or dependency relationships.
- Selecting a node should display its imports, functions, classes, and summary.
- Large graphs should support filtering or focus mode.

```

{
  "nodes": [
    {
      "id": "main.py",
      "label": "main.py",
      "type": "file"
    },
    {
      "id": "routes.py",
      "label": "routes.py",
      "type": "file"
    }
  ],
  "edges": [
    {
      "source": "main.py",
      "target": "routes.py",
      "relation": "imports"
    }
  ]
}

```

17. AI Integration and Prompt

The AI should explain repository information, not replace the static analyzer. The system should retrieve relevant files and symbols before sending context to the LLM.

10. Receive the user's question.
11. Identify keywords or relevant symbols.
12. Retrieve the most relevant files and analysis results.
13. Prepare bounded repository context.
14. Send the context and question to the LLM.
15. Return an answer with file references and limitations.

You are RepoLens AI, a codebase exploration assistant.

Rules:

1. Use only the supplied repository context.
2. Do not invent file paths, functions, or dependencies.
3. If context is insufficient, say what is missing.
4. Provide file paths and line numbers when available.
5. Distinguish evidence from assumptions.
6. Do not claim to have executed code unless results are provided.

User question:

{question}

Repository context:

{relevant_context}

Return:

- Direct answer
- Relevant files
- Supporting functions or symbols
- Limitations or missing context

Potential questions for the demo:

- What is the main entry point of this project?
- Where is the user authentication logic implemented?
- Which modules import the database service?
- Where should I modify the API endpoint for creating a user?
- Explain the relationship between the routes and models modules.

18. Frontend Dashboard

- Repository URL input and Analyze button.
- Loading indicator during analysis.
- Summary cards for file count, modules, imports, and parsing errors.
- Interactive architecture graph.
- Important modules and entry point list.
- Module details panel.
- Chat panel for repository questions.
- Error messages and empty states.
- Reset or analyze another repository option.

19. 8–12 Hour Execution Timeline

Time	Task	Main Owner	Output
0:00–0:45	Team setup and scope	All	Scope, repository, responsibilities
0:45–1:30	Project scaffolding	1, 2, 4	React, FastAPI, Git, environment files
1:30–3:00	Repository ingestion and AST	2, 3	First analysis response
2:30–4:00	Frontend shell	1	Dashboard with mocked data
3:00–5:00	Graph and API integration	1, 2, 3	Real analysis shown in UI
5:00–6:30	AI Q&A	2, 3	Grounded answers with references
6:30–7:30	Full integration	All	End-to-end workflow
7:30–9:00	Local demo hardening	4 + team	Stable local/LAN demo
9:00–10:00	Testing and backup	All	Critical bugs fixed
10:00–12:00	Presentation and rehearsal	All	Prepared live demo

If only 8 hours are available, remove multi-language support, embeddings, advanced change-impact analysis, and nonessential UI polish.

20. Local / LAN Deployment Plan (AWS Excluded)

AWS is intentionally excluded from this project plan to avoid cloud costs. The project should run on one laptop or on a local network during the hackathon.

- Run the FastAPI backend locally using Uvicorn.
- Run the React frontend locally using Vite.
- Use a local .env file for the AI API key.
- Configure the frontend API base URL to point to the backend machine.
- If other teammates need access, use the host laptop's local IP address and a controlled LAN setup.
- Keep a fully working local demo even if LAN access is not available.

Example backend command:

```
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

Example frontend command:

```
npm run dev -- --host 0.0.0.0
```

21. GitHub Collaboration Workflow

```
git clone <YOUR_REPOSITORY_URL>
```

```
git checkout -b feature/frontend-dashboard
```

```
git add .
git commit -m "Add frontend dashboard"
```

```
git push origin feature/frontend-dashboard
```

Suggested branches:

- feature/frontend-dashboard

- feature/backend-api
- feature/ai-analyzer
- feature/integration

Merge changes frequently and avoid having multiple members edit the same core files at the same time.

22. Live Demo Script

Problem: Explain that developers spend time understanding unfamiliar codebases.

Repository Input: Paste a prepared GitHub URL or load a local sample repository.

Analysis: Show file count, modules, imports, and entry points.

Architecture Map: Open the graph and select a module to show its relationships.

AI Q&A: Ask where authentication or a specific API endpoint is implemented.

Evidence: Show the file references and explain how the answer was grounded.

Technical Summary: Explain React, FastAPI, AST analysis, and local deployment.

Limitations: Mention supported languages, repository size limits, and static-analysis limitations.

23. Risks and Mitigation

Risk	Mitigation
LLM invents file names	Use retrieved context, evidence requirements, and limitation messages.
Large repository exceeds AI context	Limit files, chunk content, and retrieve only relevant context.
GitHub access fails	Use a prepared local sample repository or ZIP fallback.
AST parser crashes	Catch syntax errors and report skipped files.
Graph becomes cluttered	Filter nodes, show selected modules, or use a focused view.
AI API unavailable	Demonstrate static analysis and a prepared local fallback.
API key exposure	Use environment variables and never commit .env files.
Local networking problems	Run the entire demo on one laptop as a fallback.

24. Security, Ethics, and Limitations

- Analyze only repositories that the team is authorized to access.
- Do not upload private or sensitive source code to an external AI provider without permission.
- Never store API keys in the frontend or commit them to GitHub.
- Static analysis does not prove actual runtime behavior.
- Import relationships may not capture dynamic imports or runtime dependency injection.
- Entry-point detection is heuristic and should be labeled as “likely” when uncertain.
- AI answers should show evidence and acknowledge missing context.

25. Testing Checklist

- Frontend starts successfully.

- Backend /health endpoint returns success.
- A valid Python repository is analyzed.
- Invalid repository input returns a useful error.
- A file with a syntax error does not stop the complete analysis.
- Architecture graph renders nodes and edges.
- Selecting a node displays module details.
- AI answers at least three prepared questions.
- AI response displays file references.
- Missing context produces an uncertainty message.
- API key is loaded only from environment variables.
- The complete demo works locally without AWS.

26. Future Scope

- Support JavaScript and TypeScript repositories.
- Add semantic search using embeddings.
- Add change-impact analysis across modules.
- Generate onboarding documentation automatically.
- Provide pull request impact summaries.
- Add repository history and commit-level analysis.
- Support team workspaces and persistent project storage.
- Add dependency risk and outdated-library detection.

27. Conclusion

RepoLens AI is designed as a focused, practical hackathon MVP. Its main strength is the combination of static code analysis, architecture visualization, and AI-assisted repository Q&A. The team should prioritize a reliable local demonstration over a large number of incomplete features. The final product should show a clear problem, a working end-to-end workflow, transparent AI answers, and realistic limitations.

28. Final Pre-Demo Checklist

- React frontend works.
- FastAPI backend works.
- Repository analysis works on the selected sample project.
- Architecture graph renders.
- Important modules and entry points are shown.
- AI answers prepared questions.
- Answers include file references.
- Loading and error states work.
- No API keys are committed.
- Local fallback is ready.
- Every team member knows their responsibility.
- Presentation has been rehearsed.